

Laboratorio 2: Containerización de un Servidor Web

Diego Raúl Rivas Huanca

Grupo de laboratorio: B

15 de septiembre de 2026

1. Objetivo

Investigar y construir una imagen de contenedor que permita ejecutar un servidor web, comprendiendo la relación entre la aplicación, la imagen, el contenedor y los puertos del contenedor y de la máquina virtual.

2. Situación

Una aplicación web debe ejecutarse dentro de un contenedor. Se construyó una imagen mediante un **Dockerfile**, se ejecutó Nginx dentro del contenedor y se habilitó el acceso a la aplicación desde la máquina virtual a través del puerto 15619.

3. Dockerfile utilizado

Se partió de la imagen oficial **nginx:alpine** por ser una variante liviana y ampliamente documentada. Sobre ella se copió el contenido de la carpeta **site/** (HTML, CSS y JS separados en subcarpetas) al directorio que Nginx sirve por defecto (**/usr/share/nginx/html/**).

Listing 1: Dockerfile completo

```
1 FROM nginx:alpine
2
3 COPY site/ /usr/share/nginx/html/
4
5 EXPOSE 80
```

Explicación de cada instrucción:

- **FROM nginx:alpine:** define la imagen base, que ya trae Nginx instalado y configurado para arrancar en primer plano.
- **COPY site/ /usr/share/nginx/html/:** copia la página personalizada (y sus carpetas **css/** y **js/**) reemplazando la página de bienvenida predeterminada de Nginx.
- **EXPOSE 80:** documenta que el proceso dentro del contenedor escucha en el puerto 80. Es informativo: no publica el puerto por sí solo, eso lo hace el flag **-p** al ejecutar el contenedor.

4. Construcción y ejecución del contenedor

Construcción de la imagen a partir del Dockerfile:

```
docker build -t cc-lab2-nginx .
```

Ejecución del contenedor, publicando el puerto 15619 de la máquina virtual hacia el puerto 80 del contenedor:

```
docker run -d --name cc-lab2 -p 15619:80 cc-lab2-nginx
```

Verificación de que el contenedor está en ejecución:

```
docker ps
```

5. Página web personalizada

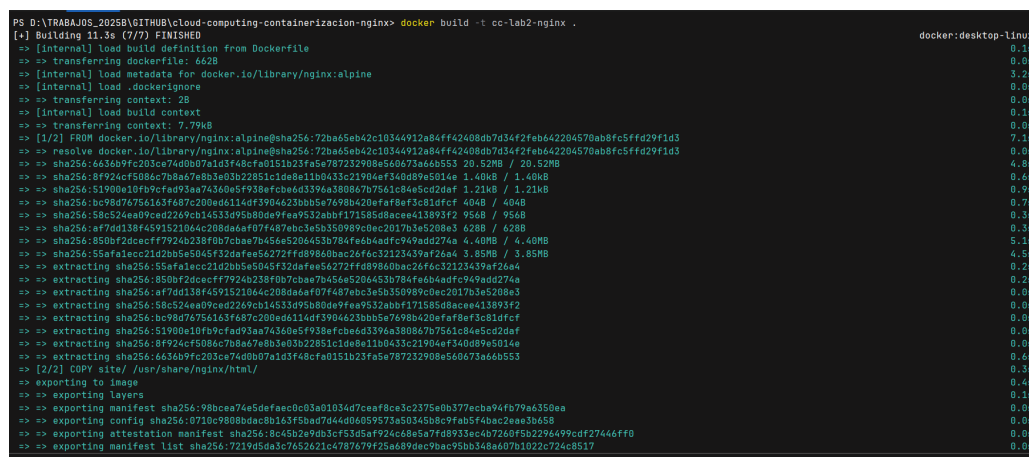
Se creó una página HTML propia (no la de bienvenida de Nginx), organizada en archivos separados:

- `site/index.html` : estructura y contenido.
- `site/css/style.css` : estilos.
- `site/js/script.js` : datos del estudiante y un fondo animado decorativo.

La página muestra el nombre del estudiante, el grupo de laboratorio y un mensaje relacionado con Cloud Computing: la idea de separar la aplicación de la infraestructura que la ejecuta, mostrando además de forma visual el mapeo de puertos VM (15619) → contenedor (80).

6. Evidencias

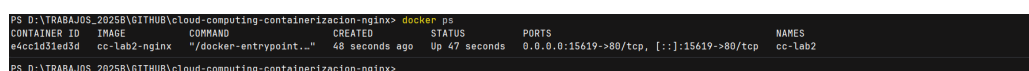
6.1. Construcción exitosa de la imagen



```
PS D:\TRABAJO_2025B\GIT\HUB\cloud-computing-containerizacion-nginx> docker build -t cc-lab2-nginx .
[+] Building 11.3s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 662B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> transferring context: 2B
=> [internal] load build context
=> transferring context: 7.79kB
[1/2] FROM docker.io/library/nginx:alpine@sha256:72ba65eb42c10344912a84ff42408db7d34f2feb642204570ab8fc5ff29f1d3
=> resolve docker.io/library/nginx:alpine@sha256:72ba65eb42c10344912a84ff42408db7d34f2feb642204570ab8fc5ff29f1d3
=> sha256:663b99fc203ce74d0d07e1d3f4dcfa051b23fa5e707232908e5d6673e6a6553 20.52MB / 20.52MB
=> sha256:8f924cf5086c7b8a67eb3e03b22851c1de8e11b0433c21904ef340d89e5014e 1.40kB / 1.40kB
=> sha256:51900e10fb9cfad93aa74368e5f938efcbe6d3396a380867b7561c84e5cd2daf 1.21kB / 1.21kB
=> sha256:bc98d76756163f687c200ed6114df3904623bbb5e7698b420efaf8ef3c81dfcf 404B / 404B
=> sha256:58c524ea09ced2269cb14533d95b00de9fea9532abbf171585d8ace413893f2 956B / 956B
=> sha256:a77ad138f453152104c208deaf07f487ebc3eb3590980dec2017b45e20a3 628B / 628B
=> sha256:85b0f2dcecf7924b238f0b7cbae7b456e5206453b784feb64dfc949add274a 4.40MB / 4.40MB
=> sha256:55afe1ecc21d2bb5e5045f32dafee56272ff489860bac26f6c32123439af26a4 3.85MB / 3.85MB
=> extracting sha256:55afe1ecc21d2bb5e5045f32dafee56272ff489860bac26f6c32123439af26a4
=> extracting sha256:85b0f2dcecf7924b238f0b7cbae7b456e5206453b784feb64dfc949add274a
=> extracting sha256:a77ad138f453152104c208deaf07f487ebc3eb3590980dec2017b45e20a3
=> extracting sha256:58c524ea09ced2269cb14533d95b00de9fea9532abbf171585d8ace413893f2
=> extracting sha256:bc98d76756163f687c200ed6114df3904623bbb5e7698b420efaf8ef3c81dfcf
=> extracting sha256:51900e10fb9cfad93aa74368e5f938efcbe6d3396a380867b7561c84e5cd2daf
=> extracting sha256:8f924cf5086c7b8a67eb3e03b22851c1de8e11b0433c21904ef340d89e5014e
=> extracting sha256:663b99fc203ce74d0d07e1d3f4dcfa051b23fa5e707232908e5d6673e6a6553
[2/2] COPY site/ /usr/share/nginx/html/
=> exporting to image
=> exporting layers
=> exporting manifest sha256:780ce574e5defac083a01034d7ceaf8c8c2375eb3777cbb94fb79e359ea
=> exporting config sha256:0710c980bdcab0143f5ba0744d06059271a50345b8c9fa05f4bac2ee3b658
=> exporting attestation manifest sha256:8c45b2e9db3cf53d5af924c48e5a7f48933ac4b7260f5b2294499cdf27446fff0
=> exporting manifest list sha256:7219d5da3c7652621c4787679f25a689dec9bac95bb348a607b1022c724c8517
```

Figura 1: Construcción exitosa de la imagen con `docker build`.

6.2. Contenedor en ejecución



```
PS D:\TRABAJO_2025B\GIT\HUB\cloud-computing-containerizacion-nginx> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
e6ccid31e63d   cc-lab2-nginx  "/docker-entrypoint..." 48 seconds ago Up 47 seconds 0.0.0.0:15619->80/tcp, [::]:15619->80/tcp  cc-lab2
```

Figura 2: Salida de `docker ps` mostrando el contenedor activo con el mapeo de puertos 15619→80.

6.3. Página accedida por el puerto 15619

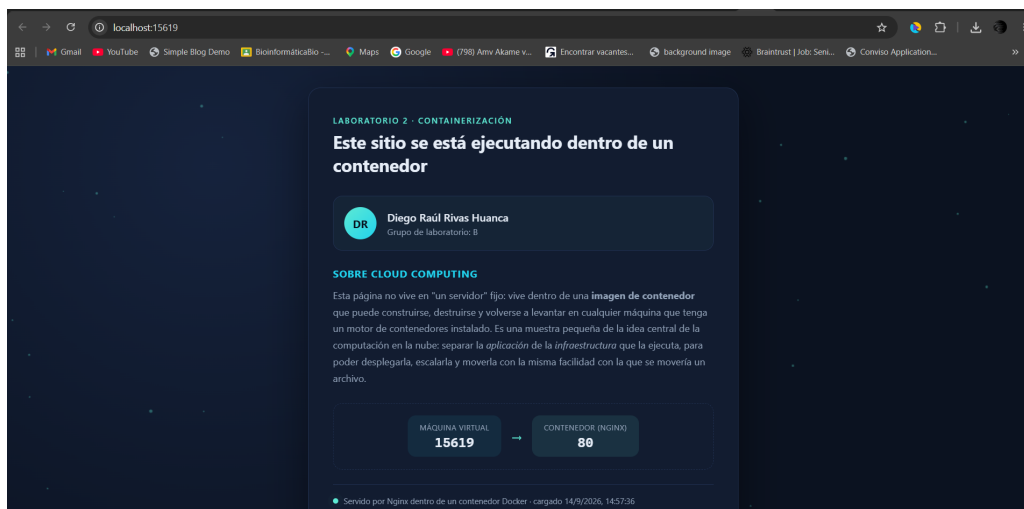


Figura 3: Navegador mostrando la página personalizada en `http://localhost:15619`.

6.4. Detención y nueva ejecución del contenedor

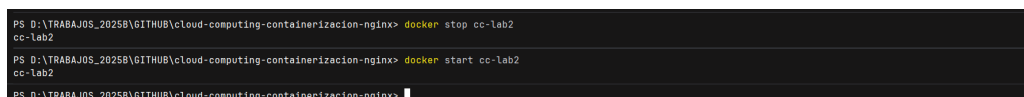


Figura 4: Secuencia `docker stop` seguida de `docker start` sobre el mismo contenedor.

Observación del requerimiento 10: al ejecutar `docker stop cc-lab2` el contenedor se detiene pero no se elimina; `docker start cc-lab2` vuelve a levantar exactamente el mismo contenedor (mismo ID, mismo filesystem) y la página vuelve a estar disponible en el puerto 15619 sin necesidad de reconstruir la imagen ni de repetir el flag `-p`, porque el mapeo de puertos quedó fijado al crear el contenedor con `docker run`.

7. Diferencia entre el puerto de la VM y el puerto del contenedor

El puerto 80 es interno: es el puerto en el que Nginx escucha *dentro* del contenedor, aislado del resto del sistema. El puerto 15619 pertenece a la máquina virtual (el host) y es el que realmente queda abierto hacia el exterior. El flag `-p 15619:80` crea un reenvío (*port forwarding*): todo lo que llega al puerto 15619 de la VM se redirige al puerto 80 dentro del contenedor. Sin esa publicación explícita, el puerto 80 del contenedor no sería alcanzable desde fuera de él, aunque Nginx sí esté escuchando allí.

8. Prompts de IA utilizados

De acuerdo al punto 6 del enunciado, se listan a continuación los prompts principales usados como apoyo para investigar sintaxis, resolver errores y comprender las instrucciones de Docker (listado completo también disponible en `prompts.md` del repositorio).

1. Construcción de un Dockerfile para Nginx sirviendo una página HTML personalizada, y ubicación correcta del COPY.

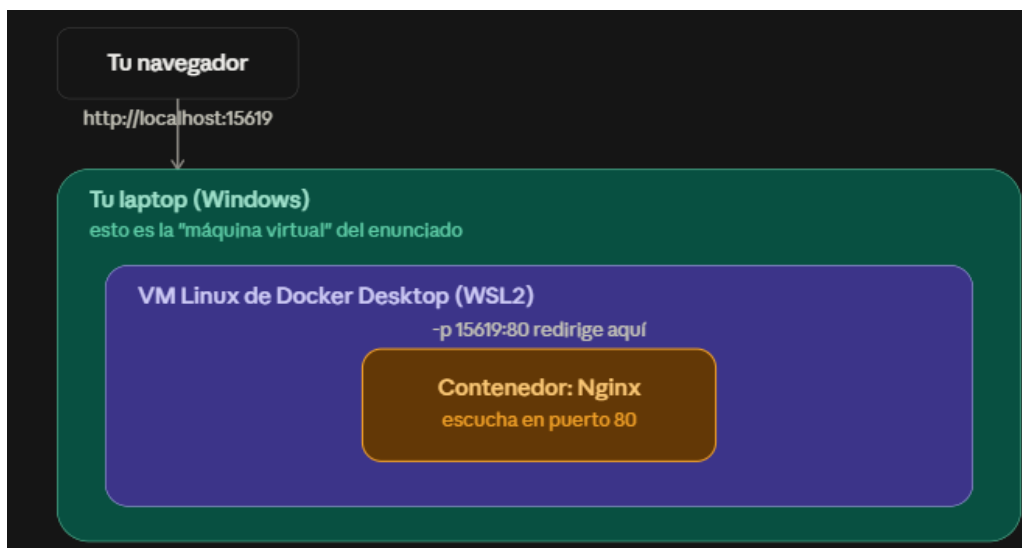


Figura 5: Arquitectura del mapeo de puertos: el navegador entra por `localhost:15619` en la máquina Windows (que cumple el rol de "VM" del enunciado), y Docker Desktop redirige esa conexión hacia el puerto 80 dentro del contenedor de Nginx.

2. Diferencia entre `EXPOSE` y el flag `-p` al publicar puertos.
3. Diferencia entre `docker stop/start` y `docker rm/run` de nuevo.
4. Organización del proyecto en carpetas separadas para HTML, CSS y JS.
5. Explicación conceptual, para el informe, de la diferencia entre el puerto de la VM y el puerto del contenedor.

9. Repositorio

Link del repositorio: <https://github.com/DiegoRivas1/cloud-computing-containerizacion-nginx.git>

10. Conclusión

El laboratorio permitió comprobar de forma práctica cómo una misma aplicación (la página web) puede empaquetarse en una imagen de contenedor reproducible y ejecutarse de forma aislada, exponiendo únicamente el puerto necesario hacia la máquina virtual. Se evidenció además que detener y volver a levantar un contenedor no reconstruye la imagen ni pierde su configuración de red, lo que ilustra la diferencia entre el ciclo de vida de una *imagen* y el de un *contenedor*.